

Ingestion — The Vertex Handler

The pivotal pipeline where an arriving vertex becomes a ledger change — verify, reach consensus, store, index, announce — in one orchestrated sequence.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 33 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

VertexHandler · Ingestion pipeline · Verify→Consensus→Store · Validation state · Partial vertices · pubsub announce · Accept/reject

Table of Contents

Chapter 33 — Ingestion: The Vertex Handler	3
33.1 Localization	3
33.2 What it does, and why it exists	4
The single chokepoint	4
Why the order matters	5
33.3 The concepts it rests on	5
33.4 The code, walked	6
A generic orchestrator first	6
The three entry points	7
Step A — Verify (and the accept / reject / already-known decision)	8
Step B — Save, then run consensus	9
Step C — Re-validate, update indexes, announce	10
Step D — Failure means crash, on purpose	11
The pipeline, in one diagram	11
33.5 How it plugs into the lifecycle	12
Who calls it	12
The convergence point	13
A note on partial vertices	13
Recap	14

Chapter 33 — Ingestion: The Vertex Handler

What you'll learn in this chapter

- What the **vertex handler** is, and why the node funnels every incoming block and transaction through this one place.
- The exact **ingestion sequence** — verify → save → consensus → re-validate → index → announce — and why that order is not negotiable.
- How the handler decides **accept** vs **reject** vs **already-known**, and what happens to a vertex whose dependencies have not all arrived yet.
- How a single failure mid-pipeline turns into a **deliberate node crash**, and why crashing is the safe choice here.
- Where this pipeline sits in the node's life: who calls it (the manager and the sync agent) and which chapters own the deep detail of each step.

This is a short chapter about a small package — two files, one of them a four-line `__init__.py`. But its size is misleading. The vertex handler is the **convergence point** of the whole node. Everything you have read about in the last several chapters — the vertex model (Ch. 25), storage (Ch. 27), indexes (Ch. 28), pub-sub (Ch. 30), verification (Ch. 31), consensus (Ch. 32) — meets here, in one method, in one fixed order. This is the box labelled "the component that runs this pipeline end-to-end" in the life-of-a-node story (Ch. 0, §0.3, Act II). We promised then that a later chapter would pay it off. This is that chapter.

33.1 Localization

The package is one of the smallest in the codebase:

```
hathor-core/
├─ hathor/
│   └─ vertex_handler/
│       ├── __init__.py
│       └─ vertex_handler.py
```

◀ YOU ARE HERE
← re-exports VertexHandler (4 lines of code)
← the whole package: class VertexHandler

`__init__.py` does nothing but lift the class up one level so the rest of the node can write `from hathor.vertex_handler import VertexHandler` instead of reaching into the inner module (`__init__.py:15`). All the substance is the single class `VertexHandler` in `vertex_handler.py:45`.

For context: the handler is built once during the builder phase (Ch. 24) and handed to the `HathorManager` (Ch. 29) as `self.vertex_handler`. It is a long-lived collaborator, created at boot and reused for the lifetime of the node.

Where it sits in the dependency graph. The vertex handler is downstream of the data model and storage, and upstream of nobody — nothing depends on it except its callers. It is glue, not foundation. It imports the verification service, the consensus algorithm, the transaction storage, the indexes, and the pub-sub bus, and it calls them in sequence. It defines no new data structures of its own. That is the shape of an **orchestrator**: a piece of code whose only job is to call other pieces in the right order.

33.2 What it does, and why it exists

The single chokepoint

Picture the two ways a new vertex¹ can appear at a running node:

1. **It was created locally.** A wallet built a transaction and submitted it; or a miner solved a block and sent it in. The manager receives it.
2. **It arrived from a peer.** The sync agent (Ch. 35) downloaded it as part of catching up, or another node relayed a brand-new transaction across the network in real time.

These are different sources, but the question they pose is identical: "Here is a candidate block or transaction. Should it become part of my ledger, and if so, what changes?" The answer requires the same work every time — check it is valid, decide what it does to canonical history, write it down, update the lookup tables, tell everyone. There is no reason to write that work twice, and several reasons not to.

So the node does it **once, in one place**: the vertex handler. Every path that wants to add a vertex to the ledger routes through this class. This is a deliberate design choice with a name — a chokepoint². Concentrating the logic here means there is exactly one definition of "what it means to ingest a vertex," exactly one place where the order of operations is decided, and exactly one place to look when something goes wrong. If verification, consensus, and storage updates were scattered across the manager, the sync agent, and the mining code, those three would inevitably drift out of sync with each other, and a vertex accepted by one path might be handled subtly differently by another. The chokepoint forbids that.

Why the order matters

The handler does not just call verification, consensus, storage, indexing, and pub-sub. It calls them **in a specific order**, and the order is load-bearing:

- **Verify before consensus.** Consensus reasons about weight, conflicts, and which history wins. It assumes the vertex is well-formed — valid signatures, sufficient proof-of-work, no internal double-spend. Running consensus on a malformed vertex would be meaningless at best and corrupting at worst. So the vertex is fully checked first.
- **Save before consensus.** Consensus walks the DAG, reading and updating the metadata of the new vertex and its neighbours. For the new vertex to participate in that walk, it must already be in storage, registered as a child of its parents. So the handler writes it to disk before asking consensus to run.
- **Consensus before announce.** The events the handler publishes — "a new transaction was accepted" — are promises to the rest of the system (the event queue, the WebSocket feed, the mining service, wallets) that this vertex is now part of the ledger and its consensus status is settled. Announcing before consensus has run would broadcast a half-truth: subscribers might react to a vertex that consensus is about to void.

So the canonical order is: **verify → save → consensus → index → announce**. The rest of this chapter is that sentence, expanded.

Recap — the ingestion pipeline (full picture: this chapter). From Chapter 0, Act II: every vertex, however it arrives, flows through **verification → consensus → store + index + announce**. The vertex handler is the code that runs exactly that pipeline. Verification answers "is this valid on its own?" (Ch. 31). Consensus answers "does this change which history is canonical?" (Ch. 32). Storage/index/pub-sub answer "record it and tell everyone" (Ch. 27, 28, 30).

33.3 The concepts it rests on

The handler is almost pure orchestration, so the heavy concepts all belong to other chapters. Here are quick recaps so you can read §33.4 without flipping back. Each ends with a pointer to its canonical treatment.

Recap — verification (full treatment in Ch. 31). Verification checks that a vertex obeys the protocol's rules: its signatures unlock the inputs it spends, its proof-of-work meets the required weight, its structure is well-formed, and it does not double-spend within itself. The `VerificationService` runs these checks. Crucially, verification comes in tiers: a basic tier that needs no dependencies, and a full tier that needs the vertex's parents and inputs to be present and themselves valid. The vertex handler calls the full tier — `validate_full ( verification_service.py:64 )`.

Recap — consensus (full treatment in Ch. 32). Consensus decides which version of history is canonical when there is any ambiguity. It does this by comparing accumulated **weight**³ and marking the losers as **voided**⁴ (recorded in each vertex's `voided_by` metadata, not deleted). Adding one vertex can ripple: a heavier block can flip the best chain, voiding some vertices and un-voiding others. The handler triggers this by calling `consensus.unsafe_update(vertex) ( consensus.py:132 )`, which returns a list of events describing what changed.

Recap — validation state and scope (full treatment in Ch. 25 & Ch. 27). Every vertex carries a **validation state** — one of `INITIAL`, `BASIC`, `FULL`, plus checkpoint variants (`validation_state.py:45`). It records how far the node has gotten in checking that vertex. `FULL` (technically `is_fully_connected()`, `validation_state.py:68`) means "fully checked, all dependencies present and valid." Separately, the storage has a **scope** that controls which validation states it will hand back. During normal operation the scope is narrowed to only fully-valid vertices — the handler asserts this with `is_only_valid_allowed() ( transaction_storage.py:402 )`.

Recap — pub-sub announce (full treatment in Ch. 30). Pub-sub is the node's internal announcement system: components publish named events to a bus without knowing who listens, and subscribers react. The handler publishes events like `NETWORK_NEW_TX_ACCEPTED ( pubsub.py:127 )` when a vertex is settled. Downstream, the event manager persists these to the durable event queue, the WebSocket feed streams them out, and the manager relays accepted vertices to peers.

33.4 The code, walked

A generic orchestrator first

Strip away Hathor and the shape is a plain pipeline orchestrator. In pseudocode, ingesting anything that must pass checks before it changes shared state looks like this:

```
def ingest(item):
    if not is_valid(item):          # 1. gate: reject bad input early
        return False
    save(item)                     # 2. persist so later steps can see it
    changes = reconcile(item)      # 3. update shared state, compute ripples
    update_lookup_tables(item)     # 4. keep derived views consistent
    announce(item, changes)        # 5. tell the rest of the system
    return True
```

Five steps: gate, persist, reconcile, re-index, announce. The vertex handler is exactly this, with `is_valid` = verification, `reconcile` = consensus, and a couple of Hathor-specific wrinkles (a re-validation pass, and a crash-on-failure policy). Keep this skeleton in mind; everything below maps onto it.

The three entry points

There is no single method called `ingest`. Instead the handler exposes **three public entry methods**, one per source, and they all funnel into one private worker. This is worth pausing on, because the naming is a little misleading — there is no method literally called `on_new_vertex`.

Entry method	Called by	Source
<code>on_new_block</code> (<code>vertex_handler.py:85</code>)	sync agent (<code>p2p/sync_v2/agent.py:617</code>)	a block from block-sync
<code>on_new_mempool_transaction</code> (<code>vertex_handler.py:117</code>)	mempool sync (<code>p2p/sync_v2/mempool.py:143</code>)	a tx from mempool-sync
<code>on_new_relayed_vertex</code> (<code>vertex_handler.py:132</code>)	manager (<code>manager.py:878</code>), real-time relay (<code>agent.py:1176</code>)	a freshly-submitted or relayed vertex

Why three? Because the three sources differ in what context they already know, not in the core work. Each entry method's real job is to assemble a `VerificationParams` object — the bundle of contextual facts verification needs, such as which block to evaluate the vertex against and which feature⁵ rules are active — and then hand off. For example, `on_new_relayed_vertex` evaluates the vertex against the current **best block** (`vertex_handler.py:140`), because a relayed real-time transaction is meant to extend the tip of the ledger. `on_new_block`, by contrast, evaluates against the block's own parent (`vertex_handler.py:87-88`), because a block arriving from sync may be deep in history, not at the tip.

All three then call the same private worker, `_old_on_new_vertex` (`vertex_handler.py:158`).⁶ That method is the real pipeline, and it is short enough to read whole:

```

def _old_on_new_vertex(self, vertex, params, *, quiet=False) -> bool:
    is_valid = self._validate_vertex(vertex, params)          # (A) verify
    if not is_valid:
        return False
    try:
        consensus_events = self._unsafe_save_and_run_consensus(vertex) # (B) save + consensus
        self._post_consensus(vertex, params, consensus_events, quiet=quiet) # (C) index + announce
    except BaseException:
        # (D) any failure here is unrecoverable: void the tx, crash the node
        self._log.error('unexpected exception in on_new_vertex()', vertex=vertex)
        meta = vertex.get_metadata()
        meta.add_voided_by(self._settings.CONSENSUS_FAIL_ID)
        self._tx_storage.save_transaction(vertex, only_metadata=True)
        self._execution_manager.crash_and_exit(...)
    return True

```

Three steps and a catch-all. Let's take them in order.

Step A — Verify (and the accept / reject / already-known decision)

`_validate_vertex` (`vertex_handler.py:187`) is the gate. It is where the handler decides whether to proceed at all, and it folds together four checks:

```

def _validate_vertex(self, vertex, params) -> bool:
    assert self._tx_storage.is_only_valid_allowed()          # scope sanity check
    already_exists = False
    if self._tx_storage.transaction_exists(vertex.hash):
        self._tx_storage.compare_bytes_with_local_tx(vertex) # same hash => must be same bytes
        already_exists = True

    if vertex.timestamp - self._reactor.seconds() > self._settings.MAX_FUTURE_TIMESTAMP_ALLOWED:
        raise InvalidNewTransaction('...in the future...') # reject: clock-skew guard

    vertex.storage = self._tx_storage
    metadata = vertex.get_metadata()

    if already_exists and metadata.validation.is_fully_connected():
        raise InvalidNewTransaction('Transaction already exists ...') # already-known
    if metadata.validation.is_invalid():
        raise InvalidNewTransaction('previously marked as invalid') # reject

    if not metadata.validation.is_fully_connected():
        self._verification_service.validate_full(vertex, params) # the real check
    return True

```

Read it as a decision tree:

- **Already-known.** If a vertex with this hash already exists and it is already fully validated (`is_fully_connected()` , `vertex_handler.py:205`), there is nothing to do — raising here is caught upstream and turns into a clean `False` return ("we already have this, didn't re-ingest"). Note the `compare_bytes_with_local_tx` call at line 191: two vertices with the same hash must have identical bytes, or the node has either a hash collision or a bug, and it wants to know immediately.

- **Reject — in the future.** A vertex whose timestamp is too far ahead of the node's clock is rejected outright (`vertex_handler.py:194`). This is an anti-abuse guard: timestamps anchor weight and difficulty calculations, so a vertex claiming to be from next year cannot be allowed to poison them.
- **Reject — previously invalid.** If the node has already seen this vertex and concluded it breaks the rules (`is_invalid()` , `vertex_handler.py:208`), it does not waste effort re-checking; it rejects again.
- **Verify — the real work.** Otherwise, if the vertex is not yet fully validated, the handler calls `validate_full` (`vertex_handler.py:213`). This is the hand-off to Chapter 31. If full validation raises a `HathorError` , the handler wraps it as `InvalidNewTransaction` (`vertex_handler.py:214-215`) and the whole ingestion ends in **reject**.

The return value is a plain `bool` : `True` means "passed the gate, proceed"; the raised exceptions are caught one level up and become `False` . So `_old_on_new_vertex` returns `True` for accepted and `False` for rejected-or-already-known — the caller cannot tell the last two apart, and mostly does not need to.

Step B — Save, then run consensus

If the gate passes, `_unsafe_save_and_run_consensus` (`vertex_handler.py:219`) does the two middle steps as one unit:

```
def _unsafe_save_and_run_consensus(self, vertex) -> list[ConsensusEvent]:
    vertex.update_initial_metadata(save=False)
    self.tx_storage.save_transaction(vertex) # (1) persist
    with non_critical_code(self._log):
        self.tx_storage.indexes.add_to_non_critical_indexes(vertex) # (2) pre-index
    return self._consensus.unsafe_update(vertex) # (3) consensus
```

The order inside is itself deliberate. `update_initial_metadata` (`vertex_handler.py:228`) registers the new vertex as a child of its parents — wiring it into the DAG. The comment in the source explains why this is done here and not earlier: the parent-child links must not be created until the vertex is known-valid, or a rejected vertex would leave a dangling child reference in storage (`vertex_handler.py:224-227`). Then `save_transaction` writes it to disk. Only now, with the vertex on disk and linked into the graph, is `consensus.unsafe_update` called (`vertex_handler.py:232`) — because consensus needs to traverse from the new vertex through its neighbours, and it can only do that if the vertex is really there.

The word **unsafe** in both method names is not decoration. It is a contract: these methods may raise, and the caller is responsible for crashing the node if they do (the docstring at `vertex_handler.py:221-222` says exactly this). We will see why at Step D.

`unsafe_update` returns a `list[ConsensusEvent]` (`consensus.py:132`). Each `ConsensusEvent` (`consensus.py:53`) is a frozen record pairing a `HathorEvents` enum value with a `kwargs` dict — a description of something consensus changed (a vertex got

voided, a block won the best chain), queued up to be published in Step C. Consensus does not publish them itself; it hands the list back so the handler can fire them in the right order, after the rest of the bookkeeping is done.

Step C — Re-validate, update indexes, announce

`_post_consensus` (`vertex_handler.py:234`) is the tail of the pipeline — the work that should happen once a vertex is fully part of the ledger:

```
def _post_consensus(self, vertex, params, consensus_events, *, quiet) -> None:
    params = replace(params, skip_block_weight_verification=True)
    assert self._verification_service.validate_full(vertex, params, init_static_metadata=False)

    self._tx_storage.indexes.update_critical_indexes(vertex) # must succeed
    with non_critical_code(self._log):
        self._tx_storage.indexes.update_non_critical_indexes(vertex) # best-effort
        self._pubsub.publish(HathorEvents.NETWORK_NEW_TX_PROCESSING, tx=vertex)
        for event in consensus_events:
            self._pubsub.publish(event.event, **event.kwargs)
        self._pubsub.publish(HathorEvents.NETWORK_NEW_TX_ACCEPTED, tx=vertex)
        self._log_new_object(vertex, 'new {}', quiet=quiet)
```

Three things happen here, and one of them may surprise you.

The surprising one: a second `validate_full`. Why validate again, having just validated in Step A? Two reasons. First, consensus may have changed the vertex's metadata (its voided status, its score), so the node re-asserts the vertex is still coherent afterwards. Second, this re-validation runs with `skip_block_weight_verification=True` (`vertex_handler.py:248`) and `init_static_metadata=False` (`vertex_handler.py:252`) — it deliberately skips the expensive weight re-check (already done) and the static-metadata initialization (already done), so it is cheaper than the first pass. It is a consistency assertion, wrapped in `assert`, not a full re-run. (The docstring at `vertex_handler.py:242-246` also notes this method can run later than ingestion for a vertex whose dependencies arrived out of order — more on that in §33.5.)

Indexes, in two tiers. `update_critical_indexes` (`vertex_handler.py:255`) updates the indexes the ledger's correctness depends on — these must succeed. Then, inside a `non_critical_code` block (`vertex_handler.py:256`), `update_non_critical_indexes` updates the convenience indexes (address history, token lists) whose failure should not crash the node — they can be rebuilt later. The `non_critical_code` context manager⁷ is the line that draws the boundary between "if this breaks, stop the world" and "if this breaks, log it and carry on."

The announce. Finally, the events fire, in order (`vertex_handler.py:259-262`):

1. `NETWORK_NEW_TX_PROCESSING` — "a new vertex is being processed, just before consensus settles" (`pubsub.py:39-41`).
2. Every `ConsensusEvent` that consensus produced — the voiding/winning ripples.

3. `NETWORK_NEW_TX_ACCEPTED` — "this vertex is now accepted into the network" (`pubsub.py:43-45`).

These are the announcements the rest of the node has been waiting for. The event manager (Ch. 30) catches them and persists them to the durable queue; the manager catches `NETWORK_NEW_TX_ACCEPTED` to relay the vertex onward to peers (`manager.py:882-883`); the mining service catches it to know a new block landed. This is the moment "data arrived" finally becomes "ledger changed, and everyone has been told."

Step D — Failure means crash, on purpose

Look again at the `except BaseException` in `_old_on_new_vertex` (`vertex_handler.py:178`). It catches everything — not just expected validation errors, but any exception at all from save, consensus, or post-consensus. And what it does is drastic: it marks the vertex voided with a special `CONSENSUS_FAIL_ID` (`vertex_handler.py:181`), saves only that metadata, and then calls `execution_manager.crash_and_exit` (`vertex_handler.py:183`) — it **takes the whole node down**.

This looks alarming. It is intentional. Recall the contract from Step B:

`_unsafe_save_and_run_consensus` may have already written the vertex to disk and begun mutating the consensus state of its neighbours before it threw. If consensus crashes halfway through a best-chain flip, the on-disk metadata is now in an unknown, possibly-inconsistent state — some vertices updated, some not. There is no safe way to continue from there. Limping on would risk acting on a corrupt ledger, which is the one thing a full node must never do (Ch. 0, §0.2: "a full node never takes another node's word for validity" — and certainly not its own corrupted state). So the handler chooses the only safe option: stop immediately, leave a marker, and let a clean restart rebuild consistent state from the storage. **Crashing is the safe failure here.** That is why the inner methods are named `unsafe`: they hand the responsibility for this crash up to `_old_on_new_vertex`, which discharges it.

The pipeline, in one diagram

Pulling Steps A–D together — and paying off the box from Chapter 0:



33.5 How it plugs into the lifecycle

Who calls it

The vertex handler is never the originator of anything — it always runs because something handed it a vertex. There are exactly two kinds of caller, both already named in earlier chapters:

- **The manager** (Ch. 29). `HathorManager.on_new_tx` (`manager.py:864`) is the front door for locally-created and submitted vertices; it calls `on_new_relayed_vertex` (`manager.py:878`) and, if the vertex is accepted and `propagate_to_peers` is set, relays it onward (`manager.py:882-883`). When a wallet submits a transaction or the local miner finds a block, this is the path.
- **The sync agent** (Ch. 35). The peer-to-peer sync code, while catching the node up to the network, calls `on_new_block` for downloaded blocks (`p2p/`

`sync_v2/agent.py:617`, `blockchain_streaming_client.py:131`),
`on_new_mempool_transaction` for mempool transactions (`mempool.py:143`), and
`on_new_relayed_vertex` for real-time relayed transactions (`agent.py:1176`).

When the node is downloading years of history from a peer, every one of those vertices passes through here.

A third, test-only caller exists — the `dag_builder` (`dag_builder/artifacts.py:88`) uses `on_new_relayed_vertex` to feed hand-built vertices into a simulated node (Ch. 43). It is the same door, used by tests.

The convergence point

Step back and notice what has assembled in this one method. Reading `_old_on_new_vertex` top to bottom, you touch:

- the **vertex model** (Ch. 25) — `BaseTransaction`, `Block`, metadata, validation state;
- **verification** (Ch. 31) — `validate_full`;
- **storage** (Ch. 27) — `save_transaction`, `transaction_exists`, scope;
- **indexes** (Ch. 28) — critical and non-critical updates;
- **consensus** (Ch. 32) — `unsafe_update` and the events it returns;
- **pub-sub** (Ch. 30) — the `NETWORK_NEW_TX_*` announcements;
- the **execution manager** — the crash-and-exit safety valve.

The vertex handler does not reimplement any of these. It sequences them. That is the entire value of the chapter: once you understand the order and the reasons for it, you understand how a node turns a pile of incoming bytes into a coherent, growing ledger.

A note on partial vertices

One subtlety ties back to Chapter 25's validation states. A vertex can arrive before all its dependencies (its parents and the inputs it spends) have arrived — common during sync, where vertices stream in roughly but not perfectly in dependency order. Such a vertex cannot be fully validated yet, because `validate_full` needs those dependencies present. In the codebase this is the `BASIC` /partial validation state (`validation_state.py:45-48`): the vertex is structurally checked but not fully connected.

The vertex handler's gate handles this gracefully. `_validate_vertex` only calls `validate_full` when the vertex is not yet fully connected and proceeds only if it succeeds (`vertex_handler.py:211-213`); a vertex still missing dependencies will fail full validation and be held back rather than ingested. When its missing dependencies finally arrive and are themselves ingested, the machinery re-runs and the vertex completes — which is exactly why `_post_consensus`'s docstring notes it "might happen later" than first receipt (`vertex_handler.py:244-246`). The handler is one stable gate

that a vertex may approach more than once, passing only when it is finally complete. The orchestration of which vertices are ready to retry lives in the sync agent (Ch. 35); the handler's job is to be the consistent, idempotent⁸ gate they all funnel through.

Recap

Step	What it does	Method (<code>vertex_handler.py</code>)	Chapter that owns the detail
Entry	Build <code>VerificationParams</code> per source, funnel to worker	<code>on_new_block:85</code> , <code>on_new_mempool_transaction:117</code> , <code>on_new_relayed_vertex:132</code>	Ch. 29 (manager), Ch. 35 (sync)
A — Verify	Dedup / future / invalid gate, then <code>validate_full</code>	<code>_validate_vertex:187</code>	Ch. 31 (verification)
B — Save	Link into DAG, persist, pre-index	<code>_unsafe_save_and_run_consensus:219</code>	Ch. 25, Ch. 27, Ch. 28
B — Consensus	Run consensus, collect change-events	<code>_unsafe_save_and_run_consensus:219</code> → <code>consensus.unsafe_update</code>	Ch. 32 (consensus)
C — Re-validate	Cheap consistency re-check post-consensus	<code>_post_consensus:234</code>	Ch. 31
C — Index	Critical (must) + non-critical (best-effort) updates	<code>_post_consensus:234</code>	Ch. 28 (indexes)
C — Announce	Publish <code>NEW_TX_PROCESSING</code> , consensus events, <code>NEW_TX_ACCEPTED</code>	<code>_post_consensus:234</code>	Ch. 30 (pub-sub)
D — Fail	Void with <code>CONSENSUS_FAIL_ID</code> , crash the node	<code>_old_on_new_vertex:178</code>	Ch. 29 (execution manager)

The vertex handler is small because it owns no concepts of its own. Its worth is in the sequence — verify before consensus before announce, save before traverse, crash before corrupt. Hold that ordering and you hold the spine of how the node grows its ledger. The one piece left unexamined is where most vertices come from in the first

place: the network. A freshly-booted node has an almost-empty database and must download years of history from strangers it does not trust. How it finds those strangers, connects to them, and streams the ledger across — feeding the vertex handler at the far end — is the subject of the next two chapters. **Chapter 34** opens the peer-to-peer layer.

1. A vertex is any node of Hathor's ledger graph — either a `Block` or a `Transaction`. The code uses "vertex" when it does not care which of the two it holds. Full treatment in Ch. 8 and Ch. 25. ↩
2. A chokepoint (or single point of control) is a deliberate design where all instances of some operation are routed through one piece of code, so the logic exists in exactly one place. The trade-off: it can become a bottleneck or a single point of failure, but it guarantees consistency. Here, consistency is worth far more than the negligible overhead. ↩
3. Weight measures how much proof-of-work a vertex represents; accumulated weight sums the weight reachable through the graph. Consensus prefers the history with the most accumulated weight. Ch. 9 and Ch. 32. ↩
4. A vertex is voided when consensus decides it is not part of canonical history — it is marked (via a `voided_by` set in its metadata), not deleted, so the node can reverse the decision if a later vertex changes the outcome. Ch. 10 and Ch. 32. ↩
5. A feature here is a protocol upgrade gated by feature activation — a mechanism where rule changes switch on according to a schedule and miner signalling. `VerificationParams` carries which features are active so verification applies the right rules. Ch. 38. ↩
6. The `_old` prefix and the docstring's "New method" are historical leftovers from a refactor; in the current code this private worker is the live ingestion path. The name is rot, not a clue — read it as "the core on-new-vertex worker." ↩
7. A context manager is the `with ...:` construct. `non_critical_code(self._log)` wraps a block so that an exception inside it is logged and swallowed rather than propagated — the boundary between "fatal if it fails" and "tolerable if it fails." ↩
8. Idempotent means running an operation more than once has the same effect as running it once. The handler's gate is idempotent in spirit: re-ingesting an already-accepted vertex is detected as "already-known" and does nothing, so a vertex can safely arrive at the gate repeatedly. ↩